COAL Assignment 2, 25K-0028 Date----------

Q1) Ax * 26, using shift addition . Ax = 3

Rough work {

```
mov ax, 3
mov bx, ax
mov cx, ax
Shl ax, 4
Shl bx, 3
Shl cx, 1
add ax, bx
add ax, cx
```

}

3 x 26 = 3 x (16+8+2) = 48+24+6 = 78

78 = 4 x 16 + 14 = 004Eh. Ax = 78, 004Eh.

Q2)

```
.data
left DWORD 02345ABCDh
Right DWORD 02345ABCDh
.code
mov esi, OFFSET left
mov ecx, 4
Loop_1:
    mov al, BYTE PTR [esi + 3]
    Shl al, 1
    rcl BYTE PTR [esi], 1
```

Brilliant

Scanned with CamScanner

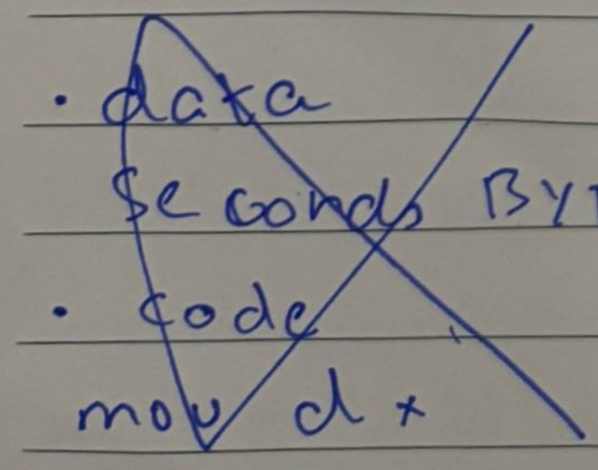
Date

```
rcl BYTE PTR [esi+1], 1
rcl BYTE PTR [esi+2], 1
rcl BYTE PTR [esi+3], 1
Loop Loop_1

mov esi, OFFSET Right
mov ecx, 4
Loop_2:
mov al, BYTE PTR [esi]
~~shrd al,1~~
~~rotate~~
shr al, 1
rcr BYTE PTR [esi+3], 1
rcr BYTE PTR [esi+2], 1
rcr BYTE PTR [esi+1], 1
rcr BYTE PTR [esi], 1
loop Loop_2
```

Q3 part a) DX: 01011 001110 11010 b

hr min seconds



Brilliant

Date----------

Q 3)

```
.data
leftval DWORD 02345ABCDh
Rightval DWORD 02345ABCDh

.code
mov esi, offset leftval
mov ecx ,2
loop_left:
    mov ax, WORD PTR [esi+2]
    shl ax, 1
    rcl WORD PTR [esi], 1
    rcl WORD PTR [esi+2], 1
    loop loop_left

mov esi, offset Rightval
mov ecx, 2.
loop_Right:
    mov ax, WORD PTR[esi]
    shr ax, 1
    RCR word ptr [esi+2], 1
    RCR WORD PTR [esi], 1
    Reloop loop_Right

exit
main proc
end main
```

Brilliant

Date _____

Q4 part a) 01011 001110 11010 b

```
.data
   Seconds BYTE ?
   value ~~word~~ 01011001110110105b
.code
main proc
   ~~mov eax, offset value~~ mov dx, 0101100111011010b
   mov ax, dx ; moving into ax
   and ax, 001Fh
   mov seconds, al

   call crlf
   ~~mov ebx, 2~~ mov ebx, 1
   call WriteBinB
   exit
main endp
end main
```

Brilliant

Scanned with CamScanner

Date

.data

4b) Minutes Byte ?

.code

mainProc → mov dx, 0101100111011101b .

Mov ax, dx

Shr ax, 5

and ax, 001Fh

mov minutes, al

→ mov ebx, 1

call writeBinB

call crlf

exit

main endp ;

end main

Date

Q5) mov dx, 9A3Fh ; dx = 9A3Fh

mov ax, 7F80h ; ax = 7F80h

clc ; CF = 0

add ax, 90EAh ; Perform +, '=' ax = 1106Ah

adc dx, 1 ; CF = 1, so add 1, dx = ~~9A36h~~ 9A41h

stc ; CF = 1

add ax, 8120h ; ax = 1918Ah

adc ~~add~~ dx, 0FFFFh ; dx = ~~1A35~~

5) DX = 9A3Fh

AX = 7F80h

CLC ; CF = 0

ADD AX, 90EAh = 1106Ah ; Discard '1' into CF

↓

AX = 1106Ah

ADC DX, 1 ; 9A3Fh + 1 + (CF = 1) = 9A41h -> DX

STC ; CF = 1

ADD AX, 8120h ; AX = 918Ah, CF = 0

ADC DX, 0FFFFh ; DX + 0FFFFh = 19A40h,

So, DX = 9A40h ↓ CF

↓

1

Final; DX = 9A40h, AX = 918Ah, CF = 1.

Brilliant

01010000

Date

Q6

a) .code
ifzero: test eax, eax
mov al, 5
cmp al, 5
jz ifzero

b) mov bx, 7
test bx, 1

c) test dx, 1000000000000000b
Or: test dx, dx
} both check if highest bit is 1.

d) test dx, 8000h

e) test bl, bl

Brilliant

Date

Q7)

ADC DX, EX ; DX + EX + 1, since CF = 1

; DX = 22

Q8 1) MOV CX, 8 X MOV ECX, 8 ; loop decrements ecx.

2) mov esi, val1 X mov esi, offset val 1 —

3) mov edi, val 2 X mov edi, offset val 2 —

4) mov ebx, offset result ; need another pointer
; to store results.

5) dec esi, dec edi, wrong. INC esi and INC edi
; we increment not decrement. Also inc ebx.

Corrected :

```
.data
val1 Qword 2040300436204 7A1h.
val2 Qword 0552103044263 0B2h
result Qword 0
.code
move cx, 8
mov esi, offset val1
mov edi , offset val2
mov ebx, offset result
```

→ Next page

Brilliant

Scanned with CamScanner

Date----------

```
clc
top:
   moval, BYTE PTR [esi]
   Sbb  al, BYTEPTR [edi]
   mov BYTE PTR [ebx], al
   inc esi
   inc edi
   inc ebx
   loop top
```

Qa)

```
.data
marks word ?
A1 byte 'A'
B2 byte 'B'
C3 byte 'C'
F Byte 'F'
Prompt byte 'Enter Marks', 0
.code
  mov edx, offset prompt
  call writestring
  call ReadDec
  mov marks, eax
```

Brilliant

Scanned with CamScanner

Include Irvine32.inc

Date

Q9)

```
.data
    marks word ?
    prompt byte 'enter marks from 0-100', 0
.code
main proc
    mov edx, offset prompt
    call WriteString
    call ReadDec
    mov marks, ax
    cmp marks, 80
    jae A1
    cmp marks, 60
    jae B1
    cmp marks, 40
    jae C1
    mov al, 'F'
    Jmp display
A1:
    mov al, 'A'
    Jmp display
B1:
    mov al, 'B'
    jmp display
C1:
    mov al, 'C'
    Jmp display
```

```
continue//
display:
    call writechar
    call crlf
    exit
main end
end main
```

Date

Q10)

```
mov eax, 3      EAX=3
mov ebx, 7      EBX=7
push eax
push ebx
```

Stack

3	
7	esp

```
pop eax         eax=7
```

3	esp

```
push 10
```

3	
10	esp

```
pop ebx         ebx=10
```

3	esp

```
pop eax         eax=3
```

final values: EAX=3, EBX=10

	stack empty

Brilliant

Date----------

Q11)

- Skipping .data , directly Writing the code :

```
.code
main Proc
mov eax, num1
mov ebx, num2
mov ecx , num3
mov edx, num4

cmp eax, ebx
JLE SecondCond
cmp ecx, edx
JE ~~ThirdCond~~ SecondCond
add eax, ecx
mov result , eax
JMP Next
SecondCond:
cmp ebx, edx
JNE ThirdCond
sub ebx , edx
~~JMP NEXT~~ mov result, ebx
JMP ~~NEX~~ Next
ThirdCond: mov result, 0
Next:
exit
end main
main endp
```

Brilliant

Date----------

Q12) • Again, skipping .data, writing code directly.

```
.code
    mov eax, op1
    mov ebx, op2
top:
    cmp eax, ebx
    JG Next
    top2:
        cmp eax, x
        JL Failed_1
        cmp eax, y
        JG Failed_1
        add z, 10
        JMP docrementing_1
    Failed_1:
        sub z, 10
    docrementing_1:
        DEC eax
        mov op1, eax
        JMP top
Next:
```

~~JMP top2~~

~~JMP top~~